SUWELA - CoreHealth Medical System

AWS Option A

Lean Production Deployment & End-to-End Pipeline

Services used, how each request travels to the end user,
how code reaches production, costs and the growth plan.

~$37	10-50	~88%	0
per month	users (initial)	cheaper than today	code changes to scale up

Contents

1. Overview
2. Architecture at a glance
3. Services used and their role
4. How a request reaches the end user (5 journeys)
5. Deployment pipeline: from code to production
6. Security layers
7. Backups and recovery
8. Monitoring and alerts
9. Monthly cost
10. Running smoothly for ~50 users
11. Growth roadmap
12. Glossary

Prepared 2 November 2026 - Planned target architecture (not yet applied). Prices are estimates excluding tax.

1. Overview

Suwela is a multi-tenant medical SaaS connecting patients, doctors, hospitals, clinics, laboratories and pharmacies. This document describes **Option A**: the smallest AWS production setup that runs the full application smoothly for the initial user base (10-50 users) while keeping medical data safe, and that can grow step by step without moving data or changing code.

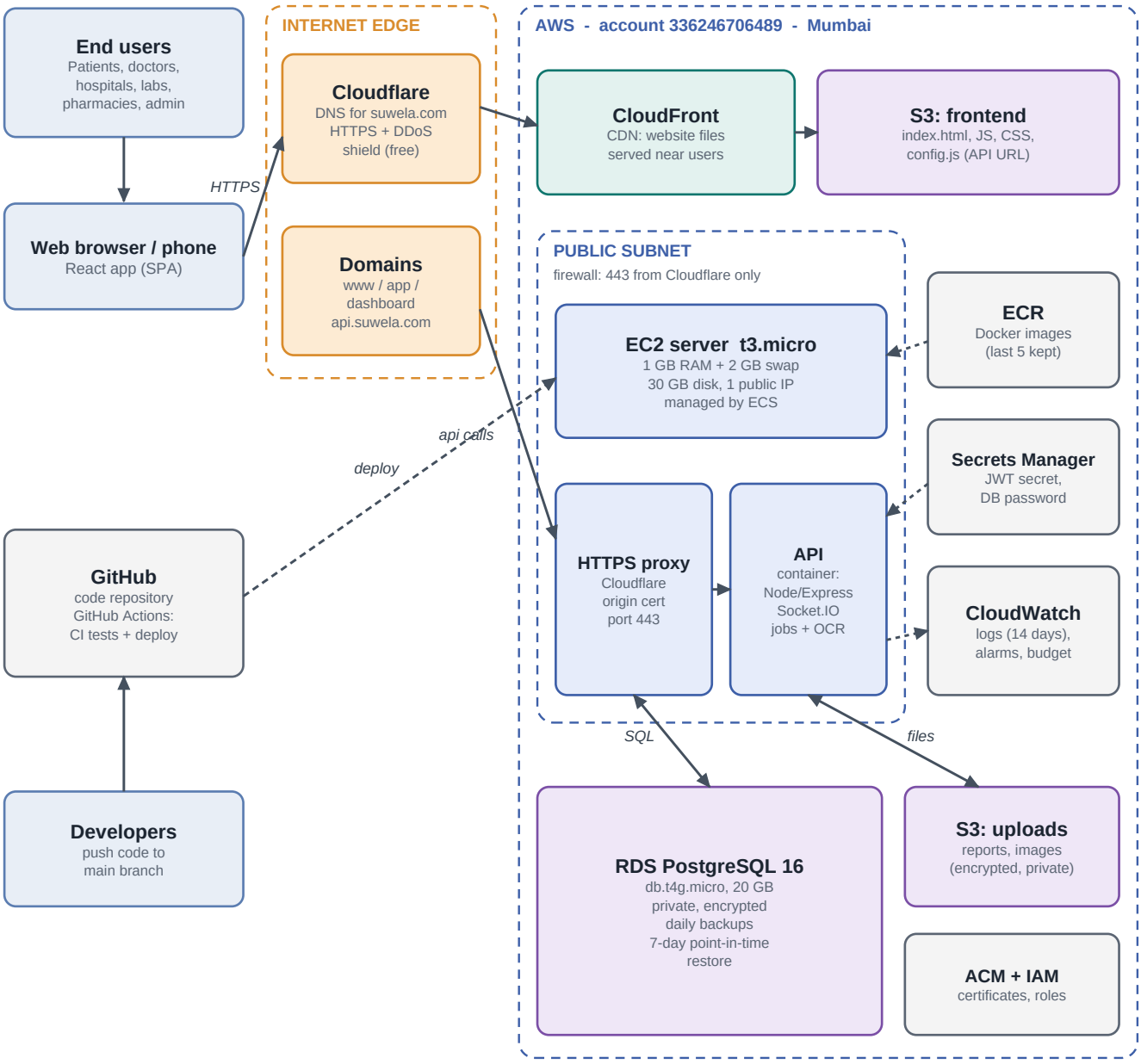
Compared with the current setup (3 servers, 2 load balancers, 2 NAT gateways and a full 24/7 staging copy, about $310/month), Option A runs **one application server and one managed database**, serves the website from a CDN, and removes the parts only needed at large scale. The monthly cost drops to **about $37**.

Layer	What runs there	Service
Edge	DNS, HTTPS certificates for visitors, DDoS protection	Cloudflare (free)
Website	The React web app (all dashboards)	S3 + CloudFront
Application	API, real-time notifications, background jobs (OCR)	1 x EC2 t3.micro managed by ECS
Data	All application data; uploaded reports and images	RDS PostgreSQL 16 + S3
Operations	Images, secrets, logs, alarms, certificates	ECR, Secrets Manager, CloudWatch, ACM, IAM
Delivery	Tests and automatic deployment	GitHub Actions

Status: this is the planned target architecture. Production currently runs the larger setup; the migration steps are in AWS_OPTION_A_LEAN_PLAN.md (about 10-15 minutes downtime, no data moved).

2. Architecture at a glance

The diagram shows every component in Option A and how they connect. Solid arrows are user traffic; dashed arrows are deployment and operations traffic.



Colour	Meaning
Blue-grey	People and their devices
Orange	Cloudflare (outside AWS)
Teal	Content delivery (CloudFront)
Blue	Compute: the server and the API
Purple	Data: database and file storage
Grey	Operations and delivery tools

3. Services used and their role

Every service below is used in Option A. Nothing else is billed. Services marked "removed" exist today but are not needed at this scale.

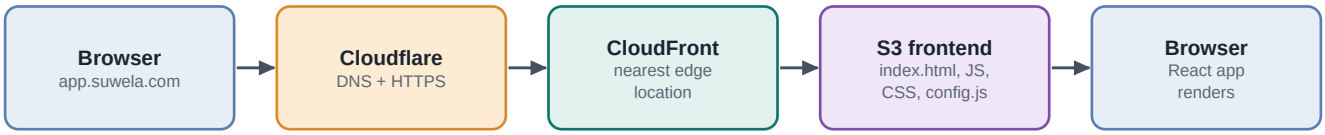
Service	Role in Suwela	Option A size	Cost / month
Cloudflare (not AWS)	DNS for suwela.com; HTTPS for visitors; hides the server IP; DDoS protection; caching	Free plan	$0
Amazon CloudFront	Delivers the website files fast from locations near users; HTTPS for www/app/dashboard	1 distribution	$0 (free 1 TB)
Amazon S3 - frontend	Stores the built website: index.html, JavaScript, CSS, images and config.js	~3 MB	~$0.10
Amazon EC2	The virtual server that runs the API container	t3.micro: 2 vCPU, 1 GB RAM + 2 GB swap	$8.20
EBS disk	Server disk: operating system, Docker images, swap	30 GB gp3	$2.75
Public IPv4	The server address Cloudflare forwards api.suwela.com to	1 address	$3.65
Amazon ECS	Starts the API container, restarts it if it crashes, swaps versions on deploy	1 cluster, 1 task	$0
Amazon RDS PostgreSQL 16	Main database: users, organisations, consultations, prescriptions, lab requests, vitals, job queue	db.t4g.micro, 1 GB RAM	$17.50
RDS storage + backups	Database disk; automatic daily backups; 7-day point-in-time restore	20 GB gp3	$2.65
Amazon S3 - uploads	Uploaded medical reports, lab results, prescription images (encrypted)	a few GB	~$0.40
Amazon ECR	Stores the API Docker images built by the pipeline (last 5 kept)	~1-2 GB	$0.20
AWS Secrets Manager	Holds the JWT signing secret and database password; injected at container start	2 secrets	$0.80
Amazon CloudWatch	API logs (14 days), alarms, AWS Budget alerts	small	~$1.00
AWS Certificate Manager	HTTPS certificates for CloudFront	-	$0
AWS IAM	Permission roles: server role (S3, secrets), deploy user for GitHub Actions	-	$0
GitHub Actions (not AWS)	Runs tests and deploys automatically	included in GitHub plan	$0
Total			**~$37**

Removed from the current setup: Application Load Balancers (~$40), NAT Gateways (~$84), 5 extra EC2 servers, 5 extra public IPs, the always-on staging environment and the larger database size. Redis (ElastiCache) stays unused: it is only needed with more than one API server.

4. How a request reaches the end user

Five everyday journeys, step by step, from the user tapping something to the result appearing on screen.

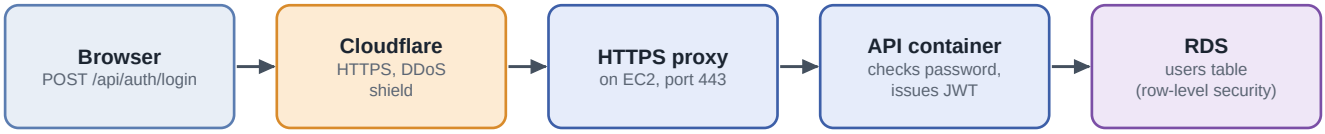
4.1 Opening the website (any dashboard)



1. The user types **www/app/dashboard.suwela.com**. Cloudflare answers the DNS lookup and secures the connection with HTTPS.
2. The request goes to **CloudFront**, which serves the files from a nearby edge location. If they are not cached yet, it fetches them once from the **S3 frontend bucket**.
3. The browser loads the React app. It reads **config.js**, which tells it the API address (https://api.suwela.com). This is why one build works for every environment.
4. The correct dashboard (patient, doctor, hospital, clinic, lab, pharmacy or admin) is shown from the user's role after sign-in.

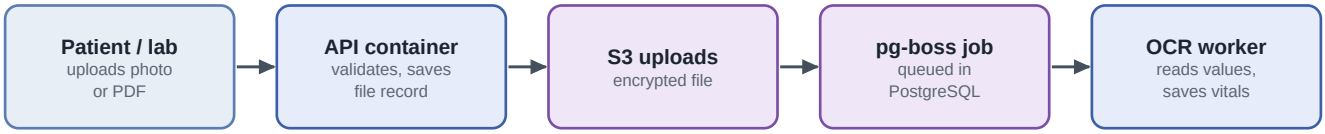
No application server is involved in loading the website, so it stays fast and nearly free even under load.

4.2 Signing in and every API call



1. The app sends the email and password to **api.suwela.com**. Cloudflare forwards it over HTTPS to the server's public IP; the server firewall accepts only Cloudflare addresses on port 443.
2. A small **HTTPS proxy** on the EC2 server (Cloudflare origin certificate) passes the request to the **API container** (Node.js/Express).
3. Security middleware runs first: Helmet security headers, CORS (only Suwela domains), and **rate limits** against password guessing.
4. The API checks the password hash (bcrypt) in **RDS PostgreSQL** and returns a signed **JWT token**. The signing secret comes from **Secrets Manager**.
5. Every later request carries the token. The API checks the role and runs database queries as that user, so PostgreSQL **row-level security** lets each user see only their own data.

4.3 Uploading a report and reading vitals (OCR)



1. The patient (or a lab) uploads a report. The API checks file type and size and stores the file in the encrypted **S3 uploads bucket**; the record goes to RDS.
2. The API replies immediately ("reading...") and puts an **OCR job** in the queue. The queue (pg-boss) lives inside PostgreSQL, so no extra queue service is paid for.
3. The worker inside the same container picks up the job: it cleans up the image (enlarge, contrast, sharpen, remove shadows), reads it several ways with Tesseract and votes on each value.
4. Found values (blood pressure, sugar, cholesterol, etc.) are saved as vitals. Failed jobs are retried automatically.
5. The patient gets a notification ("12 values added" or "could not read, try a clearer photo") and the report shows its reading status.

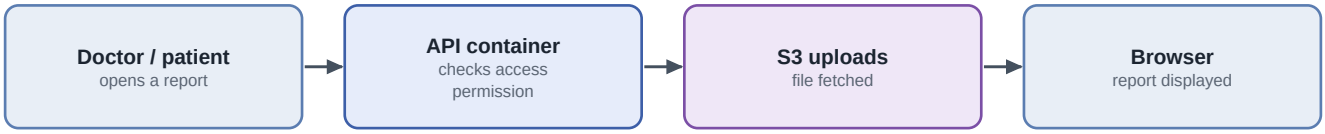
In Option A jobs run one at a time inside the API container, with 2 GB swap as a safety net. If OCR becomes slow as usage grows, the same code can run as a separate worker process (QUEUE_WORKER_MODE=separate).

4.4 Real-time notifications

1. When something happens (a lab finishes a report, a hospital certifies a doctor, a prescription is dispensed), the API saves a notification in RDS.
2. The API pushes it instantly to the right user over their open **Socket.IO WebSocket** connection, carried through Cloudflare.
3. The bell icon updates without refreshing. Clicking the notification opens the correct page for that user's role.
4. If the user is offline, the notification waits in the database and appears at next sign-in.

With a single server, all live connections are on one machine, so Redis is not needed. When a second server is added, the built-in Redis adapter is switched on.

4.5 Viewing an uploaded file

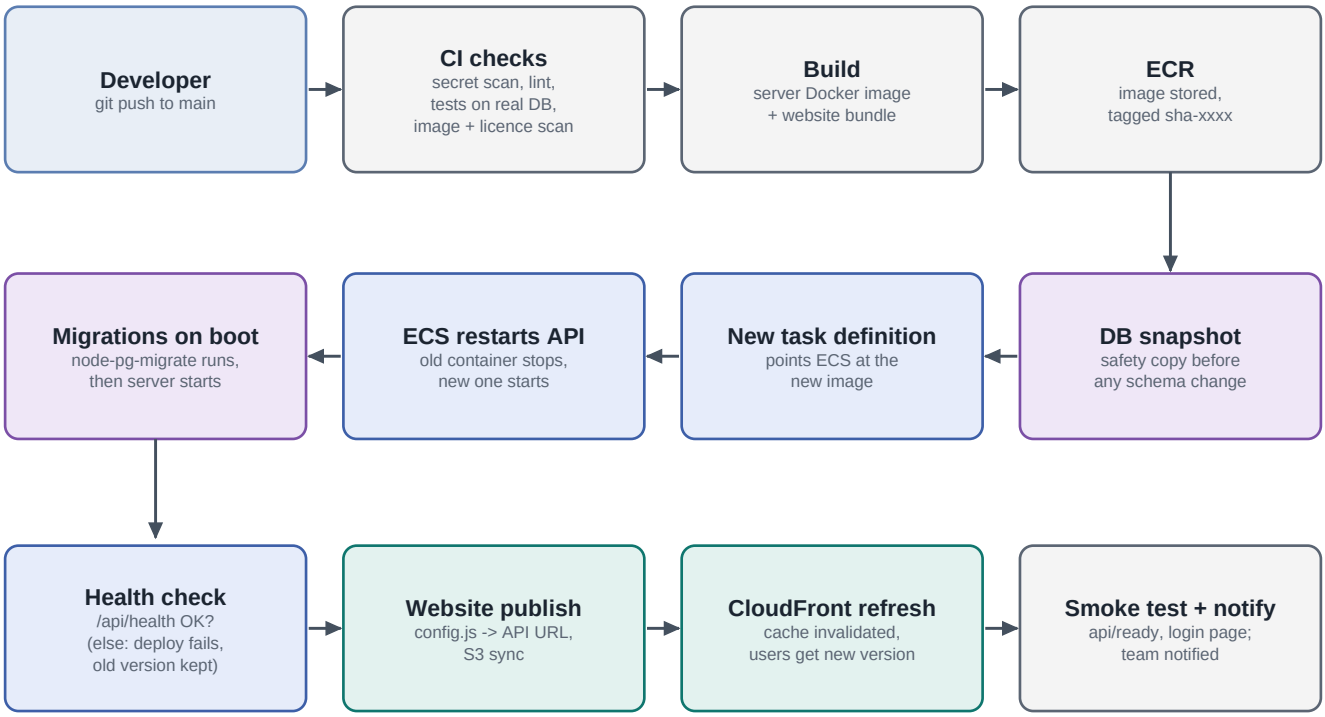


1. A doctor opens a patient's report. The API first checks permission: the patient must have granted that doctor access.
2. Only then is the file read from the private S3 uploads bucket and sent to the browser. The bucket is never public.

5. Deployment pipeline: from code to production

Every change reaches users through the same automated pipeline in GitHub Actions. Nobody uploads files to servers by hand.

Each step must succeed before the next starts. Total time ~10-15 min; API downtime ~30-60 s.



Stage 1 - Quality checks (CI)

Check	What it protects against
Secret scan (gitleaks)	Passwords or keys accidentally committed to the code
Lint and TypeScript build (server + client)	Broken code and type errors
Migrations on a real test database	Database changes that would fail in production
Server tests incl. row-level security tests	One user seeing another user's medical data
Job-queue smoke test as the app's own DB role	Background jobs failing because of missing permissions
Client tests and production build	Broken pages and dead links
Dependency audit and licence check	Known critical vulnerabilities; licences not allowed for commercial use
Docker image scan (Trivy)	Vulnerable operating-system packages in the images

Stage 2 - Build

The server is built once into a Docker image tagged with the commit (for example **sha-3a594adc90b9**) and pushed to **ECR**. The website is built once into static files. The same build is used everywhere; nothing is rebuilt for production.

Stage 3 - Release to production

1. **Database snapshot** - a manual RDS snapshot is taken first, so any schema change can be undone.
2. **New ECS task definition** - registered with the new image, with secrets and settings unchanged.

3. **Container swap** - ECS stops the old API container and starts the new one on the single server. This is the ~30-60 second downtime in Option A.
4. **Migrations on boot** - the container runs pending database migrations (node-pg-migrate) before starting the API. If a migration fails, the server does not start and the deploy fails.
5. **Health check** - ECS checks /api/health. The pipeline waits until the service is stable; if not, the deploy is marked failed.
6. **Website publish** - config.js is set to the production API address, files are synced to the S3 frontend bucket and CloudFront's cache is refreshed.
7. **Smoke test and team notification** - the pipeline calls api.suwela.com/api/ready and the login page, then reports success or failure.

Tip: trigger production deploys in low-usage hours (late night Sri Lanka time) so the short restart goes unnoticed. When zero-downtime deploys are needed, add a load balancer and a second server (Growth Stage 4) - the pipeline already supports rolling deploys.

6. Security layers

Medical data is protected by several independent layers, so one mistake does not expose data.

Layer	Protection	Where
Network edge	DDoS protection, HTTPS for every visitor, real server IP hidden	Cloudflare
Server firewall	Only HTTPS (443) from Cloudflare IP ranges; SSH closed; admin access via AWS SSM only	EC2 security group
Database network	Database has no public access; only the API server can connect	RDS security group
Encryption in transit	HTTPS end to end; TLS between API and database	Cloudflare, proxy, RDS
Encryption at rest	Database, backups, disks and S3 files are encrypted	RDS, EBS, S3
Secrets	JWT secret and DB password are never in code; injected at start	Secrets Manager
Permissions	Server role can only reach its own buckets and secrets; deploy user only deploys	IAM
Application	Password hashing, JWT sessions, role checks, rate limits, security headers, CORS	API (Express)
Data isolation	Row-level security: each user sees only data they are allowed to	PostgreSQL
Approvals	Doctors certified by their hospital, then approved by the Suwela admin; patients grant record access	Application workflow
Supply chain	Secret scanning, dependency audit, image vulnerability scanning on every change	GitHub Actions

7. Backups and recovery

What can go wrong	How it is handled	Recovery time
API container crashes	ECS restarts it automatically	1-3 minutes
Server fails	ECS / Auto Scaling replaces the server and starts the container	~5-10 minutes
Bad release	Redeploy the previous image tag from ECR	~10 minutes
Bad database migration	Restore the pre-deploy snapshot	~20-30 minutes
Data accidentally deleted	Point-in-time restore to any second in the last 7 days	~20-30 minutes
Uploaded file deleted	S3 versioning (recommended to enable) keeps previous copies	minutes
Data-centre outage	Not covered in Option A (single-AZ). Covered by Multi-AZ in Growth Stage 4	until AWS recovers

8. Monitoring and alerts

Problems should reach the team before users notice them. All of these fit in AWS's free monitoring allowance.

Alert	Trigger	Action
Site down	api.suwela.com/api/ready fails (external uptime monitor)	Check ECS service and logs
API restarted	ECS task stopped or unhealthy	Read CloudWatch logs for the cause
Server busy	EC2 CPU > 70% for 15 minutes	Consider t3.small (Stage 2)
Memory pressure	Memory > 80% or swap in regular use	Move to t3.small (Stage 2)
Database busy	RDS CPU > 70%, connections > 60	Move to db.t4g.small (Stage 3)
Disk filling	RDS free storage < 3 GB	Increase storage (no downtime)
Cost	AWS Budget reaches $45 or $60 in a month	Review the Billing console

9. Monthly cost

Item	Current setup	Option A
EC2 servers	~$98 (6 x t3.small)	~$11 (1 x t3.micro + disk)
RDS databases	~$57 (prod + staging)	~$20 (micro + storage)
NAT Gateways	~$84	$0
Load Balancers	~$40	$0
Public IPv4 addresses	~$22	~$4
S3, CloudFront, ECR, Secrets, Logs	~$9	~$2.50
Total per month	~$310	~$37 (about LKR 11,000)
Total per year	**~$3,720**	**~$444**

Saving: about $273 per month (about 88%). After the first month, a 1-year no-upfront Savings Plan (EC2) and Reserved Instance (RDS) can bring Option A to about $27-30 per month. Tax (VAT/GST) is extra depending on the billing country.

10. Running smoothly for ~50 users

Factor	Expected at 50 users	Option A capacity
People online at once	~5-15	Comfortable for one container
API requests	a few per second at peak	hundreds per second possible
API memory	~300-500 MB	1 GB RAM + 2 GB swap
Database connections	~10-20	~80+ supported
Database size	well under 1 GB in year one	20 GB, can grow without downtime
Report reading (OCR)	occasional	background queue, one job at a time
Real-time notifications	all users on one server	no Redis needed

What we accept in exchange for the low price

Trade-off	Effect	Why it is acceptable now
One server	A crash means 1-3 minutes until ECS restarts the API	Rare and short; users can retry
No load balancer	Each deploy has ~30-60 s downtime	Deploy at night
Smaller memory	Very large scanned PDFs read more slowly	One-click upgrade to t3.small (+$8)
Single-AZ database	Data-centre outage stops the site until AWS recovers	Backups keep data safe; Multi-AZ added at Stage 4
No 24/7 staging	No permanent test copy	Test locally; start staging from Terraform before big releases

Recommended habits

1. Deploy during low-usage hours.
2. Take a manual database snapshot before risky releases (the pipeline also does this automatically).
3. Keep the CloudWatch alarms and the AWS Budget alert switched on.
4. Watch memory and swap for the first weeks; upgrade to t3.small if swap is used regularly.
5. Set up email (SES or an SMTP provider) before real patients sign up, so verification and "forgot password" work.
6. Enable S3 versioning on the uploads bucket.

11. Growth roadmap

Each stage is a size or setting change in Terraform. No code changes and no data migration. Move up when the trigger signs appear, not before.

Stage	Users	Trigger signs	Change	Cost / month
1 Start (Option A)	10-50	-	1 x t3.micro + RDS db.t4g.micro	~$37
2 Comfortable	50-150	OCR slow, memory > 80%, swap in use	Server -> t3.small (2 GB)	~$45
3 Growing	150-300	RDS CPU > 60% at peak, slow pages	Server -> t3.medium (4 GB); DB -> db.t4g.small	~$80
4 Reliable	300-700	Deploy downtime is a problem; hospitals need uptime commitments	Load balancer + 2nd server (zero-downtime deploys, failover); RDS Multi-AZ	~$170
5 Scaling	700-2,000	Traffic spikes, OCR queue backlog	Auto Scaling (2-4 servers); Redis for real-time; DB -> db.t4g.medium; separate OCR worker	~$300-400
6 Enterprise	2,000+	Hospital contracts, audits, large data	Private subnets + VPC endpoints, WAF, read replica, permanent staging, 35-day backups, Savings Plans	$500+

When	Add	Why
Before real patients sign up	Email (SES or SMTP provider)	Email verification and password reset
Before real patients sign up	CloudWatch alarms + Budget alert	Know about problems and costs first
~50+ active users	1-year Savings Plan / Reserved DB	~30% cheaper for the same resources
First hospital contract	RDS Multi-AZ, longer backups	Survive a data-centre outage
~300+ users	Load balancer + 2 servers	Zero-downtime deploys and failover
More than one API server	Redis (ElastiCache)	Real-time notifications across servers (already supported in code)
Compliance audits	WAF, CloudTrail retention, private subnets	Security posture hospitals and regulators expect

12. Glossary

Term	Meaning
AWS	Amazon Web Services, the cloud provider hosting Suwela
Region / AZ	Region = a group of data centres (Mumbai). AZ (Availability Zone) = one data centre in it
EC2	A virtual server rented by the hour
ECS	The service that runs and restarts Docker containers on EC2
Container / Docker image	A packaged copy of the application that runs the same everywhere
RDS	Managed PostgreSQL database: AWS handles backups, patching and restore
S3	File storage for the website files and uploaded reports
CloudFront	CDN that delivers website files quickly worldwide
CDN	Content Delivery Network: copies of files kept near users
ECR	Private store for Docker images
Secrets Manager	Safe storage for passwords and keys
CloudWatch	Logs, metrics and alarms

Term	Meaning
IAM	AWS permissions: who and what can access which resources
JWT	Signed sign-in token the browser sends with every request
Row-level security	Database rule that filters every query to the rows the user may see
pg-boss	Background job queue stored inside PostgreSQL (used for OCR)
OCR	Reading text and numbers from images of reports
Socket.IO / WebSocket	Always-open connection used to push notifications instantly
Migration	A versioned database change applied automatically on deploy
NAT Gateway / Load Balancer	Large-scale networking components removed in Option A
Multi-AZ	A standby database copy in a second data centre